

COMSATS University Islamabad — Lahore Campus

Department of Computer Science
CSC241 Object Oriented Programming (Java)
Theory Assignment 1

Object Oriented Analysis of a Smart Campus Allocation System

Student Name: Muhammad Saoud Aziz

Registration No.: CIIT/SP26/BAI-036/LHR

Section: SP26-BAI-A

Course: CSC241 – Object Oriented Programming (Java)

Instructor: Dr. Muhammad Shahid Bhatti

Due Date: 26 September 2026

AI Disclosure: This assignment was prepared with the assistance of Claude (Anthropic) as a writing and structuring tool. All design decisions, reasoning, and justifications reflect the author's own understanding of the problem. The author takes full responsibility for every claim, model, and conclusion presented herein.

Task 1 — Stakeholder and Requirement Analysis [1.5 marks]

1.1 Stakeholders and Their Needs

- **Student:** Needs fair, transparent access to learning spaces; expects to see real-time availability without exposing whose other bookings are confirmed or for what purpose.
- **Faculty Member:** Needs priority reservation for scheduled teaching and urgent academic activities such as make-up lectures or emergency seminars. Must be able to view room capacity, equipment, and current status quickly.
- **Administrator:** Needs a high-level dashboard showing utilisation rates, approval queues, and emergency event management. Approves or rejects requests that exceed automated rules and manages room categorisation.
- **Maintenance Staff:** Needs to immediately mark a room as unsafe and block all incoming or existing reservations without navigating approval workflows. Must record reason and expected clearance date for audit purposes.
- **Security Staff:** Needs a tamper-proof log of who accessed which space and when, to support incident investigations without storing unnecessary personal data.
- **Disability Support Staff:** Needs to ensure accessible rooms remain available for users who require their features (ramps, hearing loops, adjustable furniture) without the system disclosing any individual's disability status.

1.2 Genuine Requirement Conflicts and Assumptions

Conflict 1 — Fairness vs. Faculty Priority

Students expect equitable first-come-first-served access; faculty members need guaranteed slots for scheduled teaching. These goals directly conflict when a student holds a recurring booking on a slot a faculty member needs urgently.

Assumption: Faculty reservations linked to a timetabled course override student bookings made fewer than 72 hours in advance. For all other cases, first-come-first-served applies within each user tier. This assumption is labelled explicitly in the model as a policy rule stored inside AllocationPolicy.

Conflict 2 — Room Utilisation vs. Safety Blocking

The administration targets maximum utilisation and discourages empty slots; maintenance staff must be able to close a room instantly, potentially invalidating confirmed reservations.

Assumption: A maintenance block always takes precedence over any confirmed reservation. The system must attempt automatic relocation before cancelling. If no relocation is found within a configurable grace period (default: 30 minutes), the reservation is cancelled and affected users are notified with the reason masked to 'operational requirement'.

Conflict 3 — Emergency Continuity vs. Privacy

A building closure requires the system to notify all affected users and attempt relocation across many reservations simultaneously. However, notifying users en masse risks exposing who had reservations in which building, potentially revealing attendance patterns or disability-related needs.

Assumption: Notifications are sent individually and do not include other users' names or purposes. Relocation offers are generated per reservation, and the audit log records the closure event separately from individual reservation records. Bulk processing is done server-side; no user sees another user's data.

1.3 Facts vs. Assumptions

Scenario Facts:

- Space types include classrooms, laboratories, seminar rooms, and project spaces.
- Stakeholder groups include students, faculty, administrators, security staff, maintenance staff, and disability support staff.
- A person may hold more than one role simultaneously.
- Emergency events may cancel or relocate confirmed reservations.
- The system must retain audit information and notify affected users.
- The meaning of 'fair access', limits on recurring reservations, and simultaneous-request handling are undefined in the scenario.

Introduced Assumptions:

- Faculty priority applies only to timetabled courses and only within 72 hours of the slot.
- Maintenance blocks always supersede confirmed bookings; relocation is attempted automatically.
- Individual notifications are used during emergencies to prevent cross-user data exposure.

- Accessible room availability is reserved separately from ordinary availability without recording who needs accessibility.
- An audit record stores event type and timestamp but not the requester's personal data beyond a pseudonymous user ID.

Task 2 — Object Oriented Conceptual Model [2.5 marks]

2.1 Proposed Classes

```

<<class>> Person
- personId : String
- name : String
- email : String
operations
+ getRoles() : Set
+ addRole(r:Role)
+ removeRole(r:Role)

<<class>> Role
- roleType : RoleType {enum}
- permissions : Set
- effectiveFrom : Date
- effectiveTo : Date
operations
+ hasPermission(p:Permission) : bool
+ isActive(date:Date) : bool

<<class>> LearningSpace
- spaceId : String
- type : SpaceType {enum}
- capacity : int
- isAccessible : bool
- building : String
operations
+ isAvailable(slot:TimeSlot) : bool
+ block(reason:String)
+ unblock()

<<class>> Equipment
- equipmentId : String
- description : String
- isMovable : bool
- isAvailable : bool
operations
+ allocateTo(r:Reservation)
+ release()

<<class>> Reservation
- reservationId : String
- status : ReservationStatus {enum}
- timeSlot : TimeSlot
- purpose : String
operations
+ submit()
+ confirm()
+ cancel(reason:String)
+ relocate(newSpace:LearningSpace)

```

```

<<class>> TimeSlot
- startTime : DateTime
- endTime : DateTime
- isRecurring : bool
operations
+ overlapsWith(other:TimeSlot) : bool
+ duration() : Duration

<<class>> AllocationPolicy
- priorityRules : List
- maxRecurringSlots : int
operations
+ canBook(p:Person, s:LearningSpace, t:TimeSlot) : bool
+ resolvePriority(r1:Reservation, r2:Reservation) : Reservation

<<class>> EmergencyEvent
- eventId : String
- affectedBuilding : String
- severity : String
- triggeredAt : DateTime
operations
+ getAffectedReservations() : List
+ triggerRelocations()
+ notifyAffectedUsers()

<<class>> AuditLog
- logId : String
- eventType : String
- timestamp : DateTime
- actorId : String (pseudonymous)
operations
+ record(event:AuditEvent)
+ query(filter:AuditFilter) : List

<<class>> NotificationService
- channel : NotificationChannel {enum}
operations
+ notify(person:Person, message:String)
+ notifyBulk(persons:List, message:String)

```

Figure 1 — UML class boxes for all ten proposed classes.

2.2 UML Class Diagram — Relationships and Multiplicities

Relationship	Type	Multiplicity	Rationale
Person → Role	Aggregation	1..* ↔ 0..*	A person holds zero or more roles; a role type can be assigned to many persons
Reservation → Person (requester)	Association	* ↔ 1	Many reservations per person; each reservation has exactly one requester
Reservation → LearningSpace	Association	* ↔ 0..1	Each reservation targets one space; a space may have many reservations over time
Reservation → TimeSlot	Composition	1 ↔ 1	A time slot belongs entirely to one reservation; it cannot exist independently
Reservation → Equipment	Association	* ↔ 0..*	A reservation may request zero or more pieces of equipment
AllocationPolicy → PriorityRule	Composition	1 ↔ 1..*	Rules are owned by and meaningless outside the policy
EmergencyEvent → LearningSpace (affected)	Association	* ↔ 1	An emergency targets a building; all spaces in that building are affected

EmergencyEvent → Reservation	Dependency (triggers)	0..*	An emergency event triggers state changes on many reservations
AuditLog → AuditEvent	Composition	1 ↔ 0..*	Audit events are owned by and stored inside the log
NotificationService → Person	Dependency (sends to)	1..*	NotificationService depends on Person to dispatch messages

Table 1 — UML relationships, types, and multiplicities for the conceptual model.

2.3 Responsibility Separation

The model deliberately distributes responsibilities so that no single class controls the entire system. Person only manages identity and role membership; it knows nothing about spaces or reservations. Reservation owns its own lifecycle and delegates policy checks to AllocationPolicy rather than embedding business rules inside itself. LearningSpace manages its own availability and safety status; it does not know who is booking it. EmergencyEvent is a separate class that orchestrates relocation and notification without directly modifying reservation internals—it calls Reservation.relocate() and Reservation.cancel(). AuditLog is isolated so that audit concerns cannot be accidentally bypassed by other classes. NotificationService is decoupled from all domain classes, receiving only a Person reference and a message string, which prevents notification logic from leaking into Reservation or EmergencyEvent.

Task 3 — Abstraction, Encapsulation and Invariants [2.0 marks]

3.1 Abstractions and Hidden Information

Class: LearningSpace

What it abstracts: Abstracts any physical room—classroom, laboratory, seminar room, or project space—into a uniform bookable entity with capacity, equipment, and safety status. Real-world details such as physical key codes, HVAC schedules, or exact furniture layout are not part of this abstraction.

Information that should be hidden:

- Internal safety flag (isSafe : boolean) — exposed only through block() and unblock().

Operations through which state may change:

- isAvailable(slot) — prevents double booking without exposing the internal reservation list.
- block(reason) — enforces that maintenance staff provide a reason; updates isSafe atomically.
- unblock() — only callable by maintenance or administrator roles (enforced by AllocationPolicy).

Class: Reservation

What it abstracts: Abstracts the entire lifecycle of a space request—from draft to completion or cancellation—as a single traceable object. The real-world complexity of approval workflows, conflict checks, and emergency handling is hidden behind a small state-changing interface.

Information that should be hidden:

- Requester's personal details — accessible only through a pseudonymous actorId in AuditLog.
- Internal state transitions — enforced by the state machine; callers cannot set status directly.

Operations through which state may change:

- `submit()` — validates the request against `AllocationPolicy` before moving to `PENDING`.
- `confirm()` — called only by `AllocationPolicy` after all checks pass.
- `cancel(reason)` — logs the reason in `AuditLog`; reason is never exposed to other users.
- `relocate(newSpace)` — atomically changes the associated `LearningSpace` and re-confirms.

Class: *EmergencyEvent*

What it abstracts: Abstracts a real-world crisis (fire alarm, structural concern, building-wide power failure) as a system-level event that affects multiple reservations in bulk. The complexity of iterating affected reservations, attempting relocation, and dispatching individual notifications is hidden inside this class.

Information that should be hidden:

- Internal list of affected reservations — not exposed publicly; other classes cannot iterate it directly.
- Relocation algorithm — encapsulated; callers only invoke `triggerRelocations()`.

Operations through which state may change:

- `getAffectedReservations()` — returns read-only view filtered to the caller's own reservations.
- `triggerRelocations()` — iterates and delegates; does not directly mutate `Reservation` fields.
- `notifyAffectedUsers()` — sends individual messages via `NotificationService`; no user sees another user's data.

3.2 Invariants and Encapsulation

The following invariants must hold at all times. Encapsulation enforces each by preventing direct field access from outside the owning class.

INV-1 — A `LearningSpace` in the `BLOCKED` state cannot be the target of any `CONFIRMED` Reservation.

How encapsulation enforces it: `LearningSpace.block()` iterates active reservations and calls `cancel()` or `relocate()` on each before setting `isSafe = false`. No external class can set `isSafe` directly.

INV-2 — A Reservation's status can only move forward through defined transitions (`DRAFT` → `PENDING` → `CONFIRMED` → `COMPLETED` or `CANCELLED`; `CONFIRMED` may also go to `SUSPENDED`).

How encapsulation enforces it: Status is a private field set only by state-transition methods. Invalid transitions throw an `IllegalStateException`, preventing corruption of the lifecycle.

INV-3 — A `TimeSlot` assigned to a `CONFIRMED` Reservation cannot overlap with any other `CONFIRMED` Reservation on the same `LearningSpace`.

How encapsulation enforces it: `AllocationPolicy.canBook()` checks all existing `CONFIRMED` reservations before calling `confirm()`. The check and the state change happen within the same logical transaction.

INV-4 — `AuditLog` entries are immutable once recorded; no entry may be deleted or overwritten.

How encapsulation enforces it: `AuditLog` exposes only `record()` and `query()`. There is no `delete()` or `update()` operation. The internal list of `AuditEvent` objects is held as an unmodifiable collection after each append.

Task 4 — Alternative Designs and Decision [1.5 marks]

4.1 Two Design Alternatives for Multi-Role Users

Option A — Inheritance-based roles. A class hierarchy is created: Person is the superclass; Student, FacultyMember, Administrator, MaintenanceStaff, SecurityStaff, and DisabilitySupportStaff are subclasses. Permissions are baked into each subclass. A teaching assistant who is also a student would require multiple inheritance, which Java does not support. Changing a user's role mid-semester would require constructing a new object.

Option B — Role-Object composition (chosen). Person is a single class that holds a Set<Role>. Role is an abstract class with subclasses for each role type. Permissions are stored inside each Role object and can be added or removed at runtime. A person can hold two Role objects simultaneously with no language-level conflict.

4.2 Comparison

Criterion	Inheritance-based Roles	Role-Object / Composition
Multiple simultaneous roles	Requires multiple inheritance or awkward workarounds; Java disallows it	A Person holds a Set<Role>; naturally models two roles at once
Adding a new role	New subclass + recompile entire hierarchy	New concrete Role class; no existing class touched (Open/Closed)
Changing permissions at runtime	Impossible—class is fixed at compile time	Add/remove Role objects at runtime; dynamic and safe
Coupling	High—subclasses depend on superclass internals	Low—Person knows only the Role interface
Duplicate behaviour	Common behaviour must be pushed up or duplicated across sibling subclasses	Shared behaviour lives in Role base class; reused via delegation
Ease of audit / traceability	Hard—must inspect class hierarchy	Easy—iterate Person.getRoles() and log transitions

Table 2 — Comparison of inheritance-based versus role-object approaches on six criteria.

4.3 Chosen Design and Justification

Option B (Role-Object composition) is chosen for this scenario for three reasons tied directly to the requirements. First, the scenario explicitly states that 'a person may hold more than one role, such as student and teaching assistant', which inheritance cannot model without multiple inheritance. Second, the scenario states that 'roles and permissions may change during the semester', requiring runtime mutability that a class-based hierarchy cannot provide without destroying and re-creating objects and all associated reservation history. Third, the emergency-continuity condition requires the system to quickly identify all persons with the Administrator role to dispatch relocation approvals; iterating Person.getRoles() is straightforward, whereas scanning a polymorphic hierarchy requires reflection. The composition approach is more extensible, less coupled, and directly maps to the documented stakeholder requirements.

Task 5 — Behaviour and Edge Cases [1.5 marks]

5.1 Reservation Lifecycle — State Transition Table

Current State	Event / Trigger	Next State	Action
DRAFT	User submits request	PENDING	Validate capacity & availability
PENDING	Auto/admin approval granted	CONFIRMED	Notify requester; block slot
PENDING	Request rejected	REJECTED	Notify requester with reason
CONFIRMED	Maintenance block issued	SUSPENDED	Notify user; mark room unavailable
CONFIRMED	Emergency building closure	CANCELLED	Attempt relocation; notify all affected
CONFIRMED	User cancels before deadline	CANCELLED	Release slot; update audit log
CONFIRMED	Slot time reached	COMPLETED	Log usage; release slot
SUSPENDED	Room cleared by maintenance	CONFIRMED	Restore reservation; re-notify user
SUSPENDED	Emergency closure during suspension	CANCELLED	Relocation attempted; notify affected
CANCELLED	Relocation found by system	CONFIRMED	Assign new space; notify user
CANCELLED	No relocation possible	CANCELLED	Final audit entry; close record

Table 3 — Complete state transition table for the Reservation class lifecycle.

5.2 Edge Case Responses

Edge Case 1 — Two Simultaneous Requests for the Same Space

Two users submit reservations for the same LearningSpace and overlapping TimeSlot at nearly the same time. Both enter PENDING state. AllocationPolicy processes them sequentially (using a lock or optimistic concurrency check on the space-slot pair). The first request that passes canBook() is confirmed; the second fails the overlap check and is moved to REJECTED with the reason 'slot no longer available'. If both have equal priority, the one with the earlier submission timestamp wins (first-come-first-served within the same user tier).

Responsibility placement: AllocationPolicy enforces the decision. LearningSpace provides the availability truth. AuditLog records both attempts.

Edge Case 2 — Maintenance Block After Confirmation

MaintenanceStaff calls LearningSpace.block(reason). The space finds all CONFIRMED Reservations on future slots and calls Reservation.relocate(alternativeSpace) for each, using AllocationPolicy to find a suitable alternative. If an alternative is found, the reservation moves to CONFIRMED with the new space and the user is notified. If no alternative exists within 30 minutes (configurable), Reservation.cancel(reason) is called and the user is notified. AuditLog records the block event and all resulting state changes with timestamps.

Responsibility placement: LearningSpace initiates the cascade. AllocationPolicy finds alternatives. Reservation enforces its own state transitions. NotificationService dispatches messages. AuditLog records every step.

Edge Case 3 — Emergency Cancellation or Relocation

An EmergencyEvent is created for a specific building. EmergencyEvent.getAffectedReservations() queries all CONFIRMED and SUSPENDED reservations in that building. EmergencyEvent.triggerRelocations() iterates the list and for each reservation calls AllocationPolicy to locate an alternative space (in a different building). Successful relocations call Reservation.relocate(newSpace), moving the reservation back to CONFIRMED in the new location. Where no alternative exists, Reservation.cancel() is called.

EmergencyEvent.notifyAffectedUsers() sends individual messages via NotificationService; each message names only the user's own reservation and new location. No user receives information about other users' reservations. AuditLog records the EmergencyEvent and every resulting reservation state change under the emergency event ID.

Responsibility placement: EmergencyEvent orchestrates the entire response. AllocationPolicy finds alternatives. Reservation enforces state. NotificationService sends individualised messages. AuditLog records all events under a single emergency reference for post-incident review.

Task 6 — Independent Exploration [1.0 marks]

6.1 Role-Based Access Control (RBAC)

Role-Based Access Control (RBAC) is a security model in which permissions to perform operations are assigned to roles rather than directly to individual users. A user gains access to an operation by holding a role that carries the required permission, rather than by being explicitly listed in an access control entry. This separation means that when an organisation's access policy changes, administrators modify role definitions rather than updating every user account individually (Sandhu et al., 1996). RBAC is defined by three core relationships: users are assigned to roles, roles are assigned to permissions, and permissions are granted to operations on protected resources. NIST formalises four RBAC models of increasing complexity—flat, hierarchical, constrained, and symmetric—allowing systems to adopt only the complexity they need (Ferraiolo et al., 2001).

Precise application to this model. In the Smart Campus Allocation System, RBAC is implemented through the Role and Permission classes inside the role-object composition design chosen in Task 4. The Permission enumeration contains values such as BOOK_SPACE, APPROVE_RESERVATION, BLOCK_ROOM, VIEW_AUDIT_LOG, and TRIGGER_EMERGENCY. AllocationPolicy.canBook() checks whether the requesting Person holds a Role whose permissions set contains BOOK_SPACE before creating a reservation. LearningSpace.block() first calls AllocationPolicy to verify that the caller holds BLOCK_ROOM permission, preventing students from marking rooms as unsafe. EmergencyEvent.triggerRelocations() requires TRIGGER_EMERGENCY permission, restricting that operation to Administrator-role holders only. Because permissions are stored inside Role objects rather than hard-coded into individual methods, adding a new role (such as a Graduate Teaching Assistant with limited booking authority) requires only creating a new Role object with the appropriate Permission subset—no existing class needs modification, satisfying the Open/Closed Principle.

6.2 References

- [1] Sandhu, R. S., Coyne, E. J., Feinstein, H. L., and Youman, C. E. (1996). Role-based access control models. *IEEE Computer*, 29(2), pp. 38–47. <https://doi.org/10.1109/2.485845>
- [2] Ferraiolo, D. F., Sandhu, R., Gavrila, S., Kuhn, D. R., and Chandramouli, R. (2001). Proposed NIST standard for role-based access control. *ACM Transactions on Information and System Security*, 4(3), pp. 224–274. <https://doi.org/10.1145/501978.501980>
- [3] Association for Computing Machinery and IEEE Computer Society (2020). *Computing Curricula 2020: Paradigms for Global Computing Education*. ACM/IEEE. <https://www.acm.org/education/curricula-recommendations>
- [4] Seoul Accord (2021). Section D Graduate Attributes, D.4.1 Range of Problem Solving. <https://www.seoulaccord.org/document.php?id=79>

End of Submission — CIIT/SP26/BAI-036/LHR